MILESTONE 1 IF4031 ARSITEKTUR APLIKASI TERDISTRIBUSI
SEMESTER II TAHUN 2026/2027



Kelompok 12 - JuliusAAT
JuliusSystem

Disusun oleh:

Muhammad Aufa	13523023
Julius Arthur	13523030
Ferdinand Gabe Tua Sinaga	13523051

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2026

1. Bagian Umum

1.1. Tabel Kontribusi

13523023	Implementasi Aggregator Service dari BNPB, dan canonical store
13523030	Implementasi BNPB Token Service, Client Service, serta publisher dan consumer.
13523051	Implementasi mock BMKG dan PVMBG, serta store terkait untuk masing-masing service

1.2. Deskripsi umum sistem

Sistem yang dibangun pada Milestone 1 ini merupakan platform koordinasi kebencanaan antara BMKG, PVMBG, dan BNPB, dengan BNPB berperan sebagai node koordinasi. Sistem terdiri dari beberapa layanan utama yang berkomunikasi melalui HTTP/REST dan message broker.

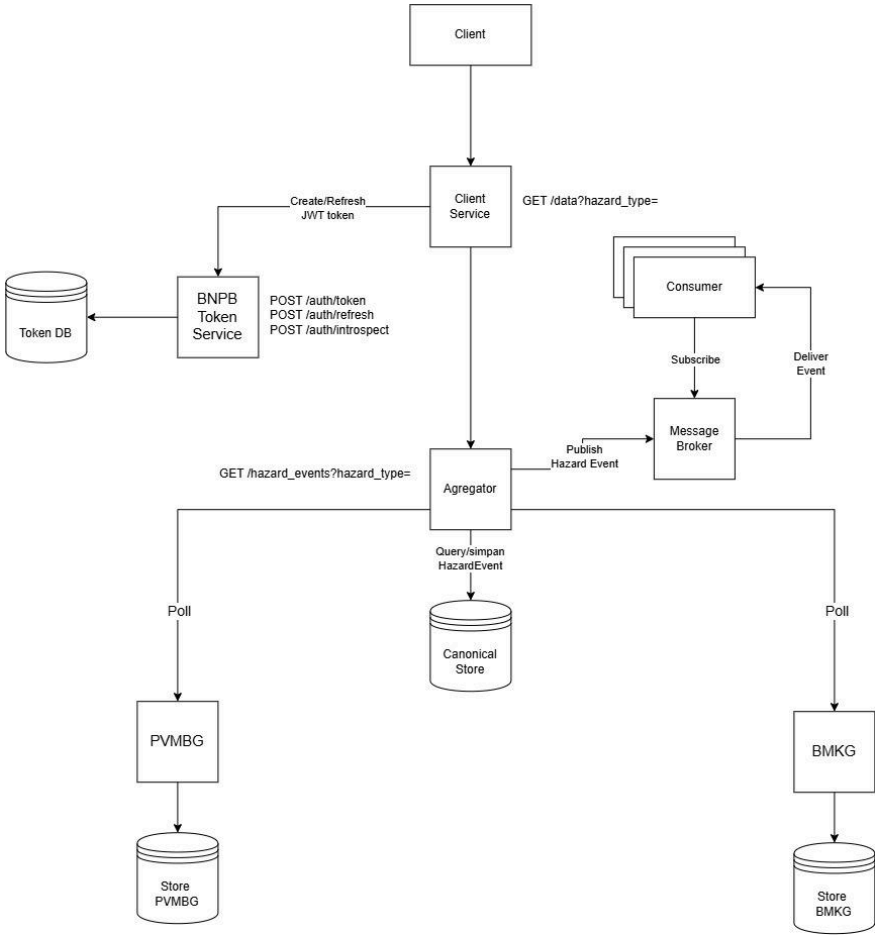
Berikut Layanan serta kontrak yang telah dibangun:

Sumber Data	
BMKG	Layanan ini menjadi sumber data untuk kejadian dalam domain gempa dan potensi tsunami. API: - GET /seismic-events - GET /tsunami-warnings - GET /health.
PVMBG	Layanan ini menjadi sumber data untuk kejadian pada domain gunung api. Layanan ini mensimulasikan infrastruktur lawas, sehingga waktu responnya lebih tinggi dan fluktuatif, yaitu dengan delay acak sebesar 500 ms hingga 3 detik. API: - GET /volcanic-reports - GET /health - POST /admin/schema-version - POST /admin/outage.

Komponen BNPB	
Aggregator	Layanan ini berperan sebagai koordinator utama yang melakukan polling secara berkala ke layanan sumber data (BMKG dan PVMBG), memetakan data ke format HazardEvent, serta bertanggung jawab memperbarui Canonical Store. Selain itu, layanan ini juga bertanggung jawab sebagai gateway bagi pihak luar meminta data HazardEvent, serta mempublikasikan setiap HazardEvent baru ke message broker. API: - GET /hazard-events?hazard_type= - GET /health
Canonical Store	Layanan yang bertanggung jawab menyimpan hasil pemetaan HazardEvent, dan hanya dapat diakses langsung oleh Aggregator. Layanan ini mendukung penyimpanan dinamis, yaitu field yang tidak dikenal atau belum memiliki padanan kanonik, seperti confidence_level, akan dimasukkan ke dalam field attributes.
Client Service	Layanan ini menjadi gateway bagi client downstream, yaitu Media/Pers, Tim Lapangan, dan BNPB Pusat, untuk mengakses data melalui Aggregator. Layanan ini bertanggung jawab memvalidasi hak akses terhadap data sesuai dengan peran masing-masing client. API: - GET /data?hazard_type= - GET /health
BNPB Token Service	Layanan ini bertanggung jawab untuk menerbitkan dan memperbarui token jwt baru untuk client downstream. Untuk memverifikasi identitas client, layanan ini memanfaaatkan client_id dan client_secretnya, lalu menerbitkan access token jwt ber TTL pendek beserta refresh tokennya jika proses verifikasi terpenuhi. Layanan ini jugalah yang membantu client service untuk memvalidasi request yang masuk kedalamnya

	API: - GET /health - POST /auth/token - POST /auth/refresh - POST /auth/introspect
Message Broker	Layanan ini berperan sebagai perantara yang menyalurkan HazardEvent dari producer (aggregator) ke consumer. Dengan adanya layanan ini, aggregator tidak perlu mengetahui consumer mana yang membutuhkan datanya.
Consumer	Layanan ini merupakan consumer yang sudah berlangganan ke MessageBroker untuk menerima HazardEvent secara otomatis, yang terdiri dari dashboard updater dan notifier.

1.3. Diagram arsitektur sistem



Sistem terdiri dari beberapa layanan utama, yaitu Client Service, BNPB Token Service, Aggregator, Message Broker, Consumer, serta layanan simulasi BMKG dan PVMBG. Setiap layanan berjalan sebagai container Docker terpisah. Komunikasi sinkron antar layanan menggunakan HTTP/REST, sedangkan distribusi event (HazardEvent) dilakukan oleh Message Broker yang menggunakan RabbitMQ.

Beberapa alur sistem dapat dibaca dibawah ini:

1. Autentikasi dan Akses Data Client Downstream
   Client downstream memperoleh access token dan refresh token dari BNPB Token Service melalui endpoint POST /auth/token. Ketika access token kedaluwarsa, client dapat memperbarui token melalui endpoint POST /auth/refresh.

   Untuk mengakses data, client mengirimkan request ke Client Service melalui endpoint GET /data dengan menyertakan access token. Client Service kemudian memvalidasi token tersebut dan memanggil endpoint POST /auth/introspect untuk memastikan bahwa token dan sesi pengguna masih aktif. Setelah proses autentikasi berhasil, Client Service akan memproses permintaan dan mengembalikan data sesuai dengan role yang dimiliki oleh client.

2. Permintaan Data Client
   Ketika terdapat permintaan data dari client melalui Client Service, Aggregator Service akan mengambil data yang diperlukan dari Canonical Store untuk kemudian diteruskan kepada client.

3. Ingest data
   Data yang disediakan kepada client berasal dari Aggregator Service yang melakukan polling secara berkala ke BMKG dan PVMBG. Data yang diterima dari kedua sumber tersebut kemudian dipetakan ke dalam format kanonik HazardEvent dan disimpan pada Canonical Store.

4. Distribusi Event
   Setiap kejadian baru dari BMKG maupun PVMBG yang telah dipetakan menjadi HazardEvent oleh Aggregator dipublikasikan ke Message Broker. Consumer berlangganan ke broker dan menerima event secara independen satu sama lain.

1.4. Daftar teknologi terpilih

Kebutuhan	Teknologi	Alasan
HTTP Service	Node.js dan Express	Ringan dan mudah diimplementasikan
Canonical Store	PostgreSQL	Persistence dan durable
Message broker	RabbitMQ	Dapat memiliki satu queue untuk setiap consumer dan routing melalui exchange
Container	Docker	Dapat terkontainerisasi
Autentikasi client & service	JWT	Stateless, dapat memodelkan expiry serta pencabutan sesi dengan introspection

1.5. Asumsi

1.5.1. Kredensial untuk akses data pada mock baik API KEY, TOKEN JWT serta SECRET semua di-*generate* sebelum service PVMBG dan BMKG dihidupkan

1.5.2. Tipe data hasil poll Aggregator ke mock diasumsikan tidak akan berubah sehingga selama data yang di-*generate* secara random pada mock valid maka insert ke kanonik store tidak akan pernah gagal

1.5.3. Reference table gunung api di Aggregator diasumsikan mencakup seluruh volcano_id yang mungkin dikirim oleh PVMBG

1.5.4. Aggregator diasumsikan dapat tetap dinyalakan walaupun BMKG/PVMBG mock belum siap, sesuai dengan tema bahwa Aggregator harus tetap berjalan walaupun salah satu sumber gagal, dan sudah memiliki penanganan try catch polling error nya.

1.5.5. Skema pada PVMBG tidak mengalami perubahan ketika endpoint POST /admin/schema-version dipanggil. Endpoint tersebut hanya mengatur agar kolom confidence_level, yang sebelumnya bernilai NULL, diisi dengan nilai Float.

1.6. Petunjuk Menjalankan Sistem

Langkah secara lengkap untuk menjalankan sistem, beserta daftar variabel konfigurasi nya, dapat dilihat pada README dari repository. Secara ringkas, cara untuk menjalankan meliputi langkah-langkah berikut:

1. Clone Repository dan masuk ke direktori proyek
2. *Copy* .env.example pada repository pada file .env yang dibuat kemudian isi variabel yang diperlukan berdasarkan daftar variabel konfigurasi nya. Khusus untuk kredensial BMKG dan PVMBG, ikuti langkah 3.
3. Jalankan kode berikut menggunakan Node.js untuk menghasilkan kredensial yang diperlukan untuk PVMBG dan BMKG. Pastikan dependensi jsonwebtoken telah tersedia di lingkungan Node.js yang digunakan.

```
const crypto = require('crypto');
const jwt = require('jsonwebtoken');

const jwtSecret = crypto.randomBytes(32).toString('hex');
const apiKeyBmkg = crypto.randomBytes(32).toString('hex');

const token = jwt.sign(
    {
        iss: 'pvmbg',
        sub: 'Aggregator',
        role: 'poller'
    },
    jwtSecret,
    {
        expiresIn: '48h'
    }
);

console.log('\n=== CREDENTIALS ===');
console.log('PVMBG_SECRET_KEY=' + jwtSecret);
console.log('BMKG_API_KEY=' + apiKeyBmkg);

console.log('\n=== JWT TOKEN ===');
console.log('PVMBG_TOKEN=' + token);
```

4. Jalankan script untuk menghasilkan public dan private key dengan `node .\bnpb-token\script\gen-keys.js`
5. Jalankan seluruh service dari sistem dengan `docker compose up –-build -d`
6. Verifikasi seluruh service telah berjalan dengan `docker compose ps`

2. Bagian Per Problem

2.1. Arsitektur dan Interoperabilitas Data

2.1.1. Rancangan Solusi

Untuk mengatasi permasalahan interoperabilitas data, Aggregator Service menggunakan pendekatan dynamic field mapping dengan kebijakan additive-only. Beberapa field utama, seperti report_id, volcano_id, alert_level, dan reported_at dari PVMBG, serta event_id, dan region_name dari BMKG, dipetakan secara eksplisit ke field kanonik HazardEvent, sedangkan field seperti depth_km, potential_tsunami, dan magnitude dari BMKG, serta eruption_count_24h dan ash_column_height_m dari PVMBG, dipetakan sebagai properti dari *dynamic field* bernama attributes. Hal ini didasarkan pada aturan pemetaan yang telah ditentukan.

Field yang tidak memiliki pemetaan akan secara otomatis juga dimasukkan ke dalam field attributes. Dengan pendekatan ini, ketika PVMBG menambahkan field baru, seperti confidence_level melalui POST /admin/schema-version, field tersebut dapat langsung diterima dan disimpan di dalam attributes tanpa memerlukan perubahan kode, restart, maupun redeploy pada Aggregator Service.

2.1.2. Alasan keputusan

Alternatif	Kelebihan	Kekurangan
Schema registry	Skema terpusat dan terversi, kompatibilitas dapat dicek secara formal	Memerlukan komponen tambahan dan koordinasi dengan PVMBG, padahal PVMBG tidak memberi tahu perubahan skema mereka
Schema evolution ala Protobuf atau Avro	Penerima versi lama tetap dapat memproses pesan versi baru tanpa gagal karena field yang tidak dikenali diabaikan	Komunikasi dengan sumber menggunakan JSON melalui REST. Field baru tetap tidak dapat dibaca sampai skema penerima diperbarui dan sistem

		di-deploy ulang
Dynamic field mapping dengan JSON additive-only (dipilih)	Paling sederhana. Field baru langsung diterima dan disimpan di attributes tanpa perubahan kode, restart, maupun redeploy	Perubahan non-aditif (hapus, ganti nama, ubah tipe, pindah ke struktur bersarang) tidak didukung

2.1.3. Batasan

Sistem hanya mendukung perubahan skema yang bersifat aditif, yaitu penambahan field baru pada payload sumber. Sistem secara sadar tidak mendukung penghapusan field yang sudah dipetakan secara eksplisit ke field kanonik, penggantian nama field tersebut (misalnya alert_level menjadi level), perubahan tipe datanya, maupun pemindahan field tersebut ke dalam struktur bersarang (misalnya alert_level dipindah ke dalam objek alert.level), tetapi hal ini tidak berlaku untuk penambahan field baru yang strukturnya bersarang tetap dapat disimpan apa adanya, misalnya ({ "meta": { "confidence_level": 0.8}}

2.1.4. Bukti Penyelesaian

2.1.4.1. HazardEvent sesuai dengan aturan pemetaan

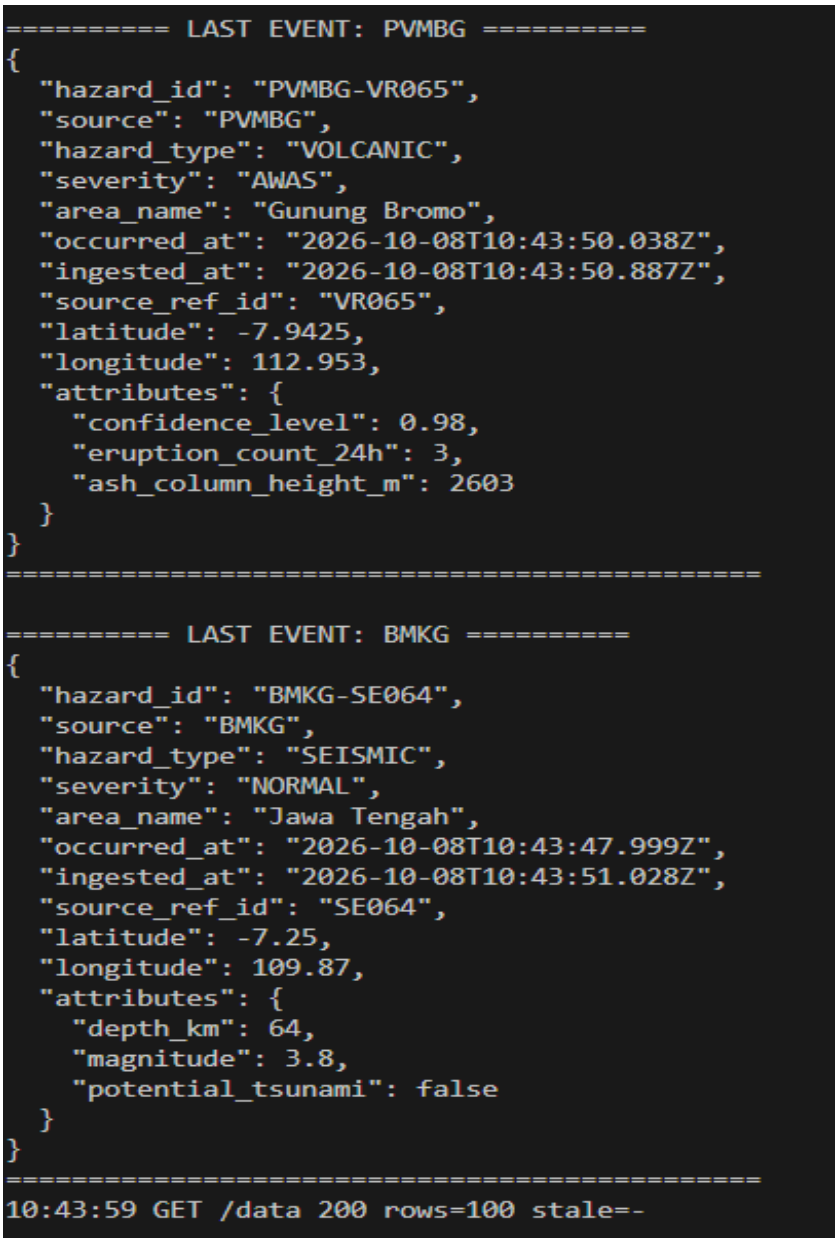

```
========== LAST EVENT: PVMBG ==========
{
  "hazard_id": "PVMBG-VR041",
  "source": "PVMBG",
  "hazard_type": "VOLCANIC",
  "severity": "WASPADA",
  "area_name": "Gunung Sinabung",
  "occurred_at": "2026-10-08T10:40:22.663Z",
  "ingested_at": "2026-10-08T10:40:23.593Z",
  "source_ref_id": "VR041",
  "latitude": 3.17,
  "longitude": 98.392,
  "attributes": {
    "eruption_count_24h": 13,
    "ash_column_height_m": 389
  }
}
============================================

========== LAST EVENT: BMKG ==========
{
  "hazard_id": "BMKG-SE042",
  "source": "BMKG",
  "hazard_type": "SEISMIC",
  "severity": "WASPADA",
  "area_name": "Kepulauan Banda",
  "occurred_at": "2026-10-08T10:40:29.611Z",
  "ingested_at": "2026-10-08T10:40:29.706Z",
  "source_ref_id": "SE042",
  "latitude": -5.83,
  "longitude": 130.21,
  "attributes": {
    "depth_km": 149,
    "magnitude": 5.84,
    "potential_tsunami": false
  }
}
============================================
```


2.1.4.2. Berhasil trigger perubahan schema pada PVMBG


```
PS D:\Tubes AAT\Tubes-IF4031-Kel12_JuliusAAT> curl.exe -X POST "http://localhost:8082/admin/schema-version" -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJwdmliZyIsInN1YiI6IkFnZ3JlZ2F0b3IiLCJyb2xlIjoicG9sbGVyIiwiaWF0IjoxNzkxNDUzMTUwLCJleHAiOjE3OTE2MjU5NTB9._yHUikTWBvzWgUi4jzE6fsMgyIkEvtM2CAEGeWdfGSw" -H "Content-Type: application/json" -d '{\"confidence\":true}'
{"status":"ok"}
```

2.1.4.3. Aggregator tidak crash dan tetap menghasilkan HazardEvent



2.1.5. Jawaban atas butir "Di laporan"

Mekanisme yang dipilih adalah *dynamic field mapping* dengan kebijakan JSON *additive-only*. Field yang telah dikenal dipetakan secara eksplisit ke field kanonik HazardEvent berdasarkan aturan pemetaan. Sementara itu, field yang tidak memiliki aturan pemetaan diteruskan apa adanya ke dalam

field attributes. Dengan pendekatan ini, ketika PVMBG menambahkan field baru, misalnya confidence_level, field tersebut dapat langsung disimpan di attributes tanpa memerlukan perubahan kode, restart, maupun *redeploy* pada Aggregator.

Mekanisme ini secara khusus mendukung perubahan skema berupa penambahan field baru, baik pada level atas payload maupun di dalam struktur bersarang. Field baru seperti confidence_level dapat diteruskan ke attributes, sedangkan field bersarang juga dapat disimpan apa adanya tanpa memerlukan perubahan pada aturan pemetaan. Sebaliknya, mekanisme ini tidak mendukung penghapusan, penggantian nama, perubahan tipe data, maupun pemindahan field yang sebelumnya telah dipetakan secara eksplisit. Penghapusan field akan menyebabkan aturan pemetaan tidak lagi menemukan nilai yang dibutuhkan untuk field kanonik, sedangkan penggantian nama, misalnya alert_level menjadi level, membuat nama field baru tidak dikenali oleh aturan pemetaan. Perubahan tipe data juga tidak ditangani karena Aggregator tidak melakukan validasi terhadap tipe isi event. Demikian pula, pemindahan field ke struktur bersarang, seperti alert_level menjadi alert.level, tidak didukung karena aturan pemetaan hanya membaca field pada posisi yang telah ditentukan. Dengan demikian, pendekatan ini memberikan fleksibilitas terhadap penambahan informasi baru, tetapi tetap bergantung pada aturan pemetaan eksplisit untuk field yang memiliki peran kanonik.

2.2. Concurrency, Availability, & Graceful Degradation

2.2.1. Rancangan Solusi

Pada problem 2 terdapat tiga masalah yang perlu ditangani, yakni Pemanggilan BMKG dan PVMBG yang sekuensial (*head-of-line blocking*), tidak ada nya rate limit permintaan (Resource exhaustion), dan efek domino dari kematian satu sumber service (Ketiadaan degradasi terkendali).

Untuk mengatasi head-of-line blocking, Proses pengambilan data dari BMKG dan PVMBG dengan proses query dari client dipisahkan pada Aggregator. Aggregator melakukan polling berkala ke PVMBG dan BMKG kemudian hasilnya disimpan ke Canonical Store. Permintaan client pada endpoint GET /hazard-events hanya membaca data dari Canonical Store saja (tidak langsung ke BMKG dan PVMBG), sehingga latensi permintaan client tidak bergantung pada waktu respons kedua sumber tersebut. Pada setiap siklus polling, pemanggilan ke BMKG dan

PVMBG dilakukan secara paralel menggunakan Promise.allSettled, tidak secara sekuensial, sehingga PVMBG yang lambat tidak menunda selesainya pengambilan data BMKG pada siklus yang sama.

Untuk mengatasi resource exhaustion, Aggregator membatasi jumlah permintaan yang dapat diproses melalui rate limiter menggunakan library express-rate-limit yang dipasang pada endpoint GET /hazard-events. Permintaan yang melebihi batas yang ditentukan akan ditolak dengan status code HTTP 429. Selain itu, jumlah koneksi ke Canonical Store dibatasi melalui connection pool pada driver PostgreSQL sehingga lonjakan permintaan tidak membuka koneksi database tanpa batas.

Untuk mengatasi ketiadaan *graceful degradation*, Aggregator mencatat status keberhasilan dan kegagalan polling terhadap BMKG dan PVMBG mock service pada tabel ingest_status dalam store-aggregator, beserta waktu polling berhasil nya. Ketika PVMBG sedang outage, permintaan data seismik tetap dilayani dengan normal karena data tersebut berasal dari BMKG yang tidak terpengaruh, sedangkan untuk permintaan data vulkanik dijawab dengan status code 200 menggunakan data terakhir yang tersimpan dari canonical store (hasil polling terakhir terhadap PVMBG sebelum outage) disertai dengan field stale_since apabila waktu sejak polling PVMBG terakhir berhasil melebihi dua kali interval polling yang dikonfigurasi.

2.2.2. Alasan keputusan

2.2.2.1. Keputusan terkait pemanggilan BMKG dan PVMBG dalam satu siklus polling

Alternatif	Kelebihan	Kekurangan
Polling sekuensial (await BMKG, baru await PVMBG tiap siklus)	Sesuai pola polling yang diwajibkan pada spek	PVMBG yang lambat menunda tersedianya data BMKG baru pada siklus yang sama, menyebabkan head-of-line blocking pada jalur ingest
Polling paralel dengan Promise.allSettled (dipilih)	kedua sumber diproses bersamaan, kegagalan satu sumber tidak memengaruhi sumber lain	Perlu penanganan hasil per sumber secara terpisah di kode

Worker thread atau proses terpisah per sumber	Isolasi penuh antar sumber data	Kompleksitas implementasi yang tinggi dan tidak diperlukan karena Node.js sudah non-blocking secara I/O pada level event loop

2.2.2.2. Keputusan terkait pembatasan jumlah permintaan bersamaan

Alternatif	Kelebihan	Kekurangan
Token bucket atau sliding window	Penolakan terdistribusi lebih merata, tidak ada lonjakan tajam di batas window	Implementasi lebih kompleks dan tidak tersedia sebagai middleware siap pakai sesederhana fixed window
Fixed window counter dengan express-rate-limit (dipilih)	Mudah dipasang sebagai middleware Express dari library express-rate-limite yang sudah ada dan cukup untuk membatasi beban pada skala ini	Berpotensi mengizinkan lonjakan permintaan di sekitar batas window karena counter direset serentak
Rate limiter terdistribusi berbasis Redis	Konsisten walaupun Aggregator dijalankan sebagai banyak instance	Menambah komponen infrastruktur baru yang tidak diperlukan, belum dibutuhkan multi instance

2.2.2.3. Keputusan terkait penanganan permintaan data vulkanik saat PVMBG outage

Alternatif	Kelebihan	Kekurangan
Status ketidaktersediaan eksplisit untuk seluruh permintaan vulkanik	Sinyal tidak tersedia sangat jelas bagi client	Client kehilangan akses ke data terakhir yang sebenarnya masih berguna untuk kondisi darurat
Penanda stale_since pada data terakhir dari Canonical Store (dipilih)	Client tetap menerima data terakhir yang diketahui, dan tetap tahu data tersebut sudah tidak terbaru	Client harus memeriksa field stale_since secara eksplisit, tidak otomatis terlihat seperti kondisi error
Circuit breaker dengan health check terpisah dan lapisan cache tambahan	Dapat menghentikan percobaan polling lebih cepat saat outage berkepanjanga n	Tidak cukup *necessary* karena Aggregator sudah mencatat status tiap sumber secara independen melalui ingest_status

2.2.3. Batasan

Implementasi Rate Limiter dan Connection pool sekarang bersifat global dalam satu proses Aggregator dan disimpan dalam memori. Hal ini tidak akan bekerja jika Aggregator kedepannya dapat dijalankan banyak *instance* karena batas yang diterapkan tidak akan terkoordinasi antar *instance* tersebut.

Siklus polling tidak memiliki mekanisme penjagaan terhadap kemungkinan *overlap*, yaitu kondisi ketika satu siklus polling belum selesai saat siklus berikutnya mulai dijalankan.

Penanda stale_since bersifat heuristik berdasarkan ambang waktu tetap, yaitu dua kali interval polling, bukan jaminan real-time atas kondisi PVMBG.

2.2.4. Bukti Penyelesaian

2.2.4.1. Load Testing

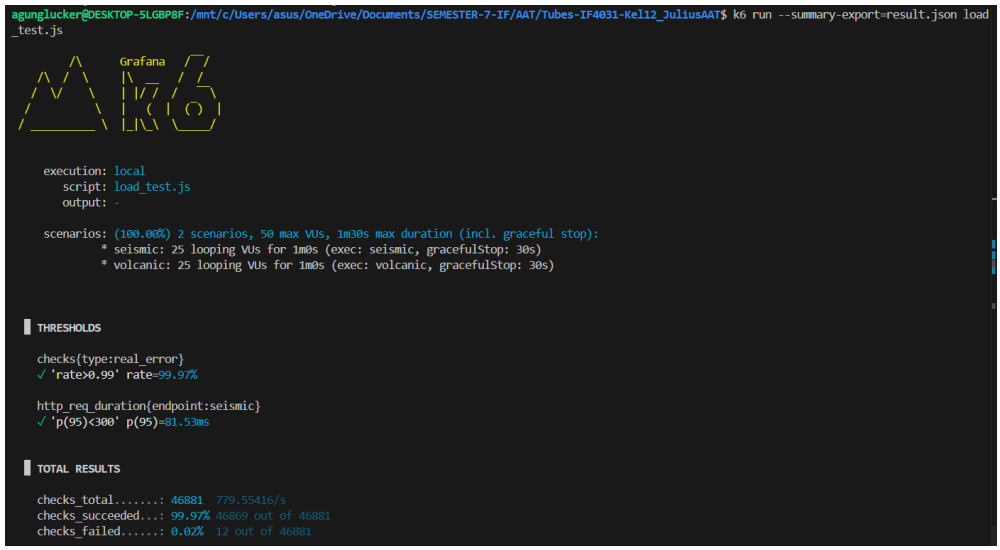
Command untuk Load env dari file .env untuk load testing (wsl)

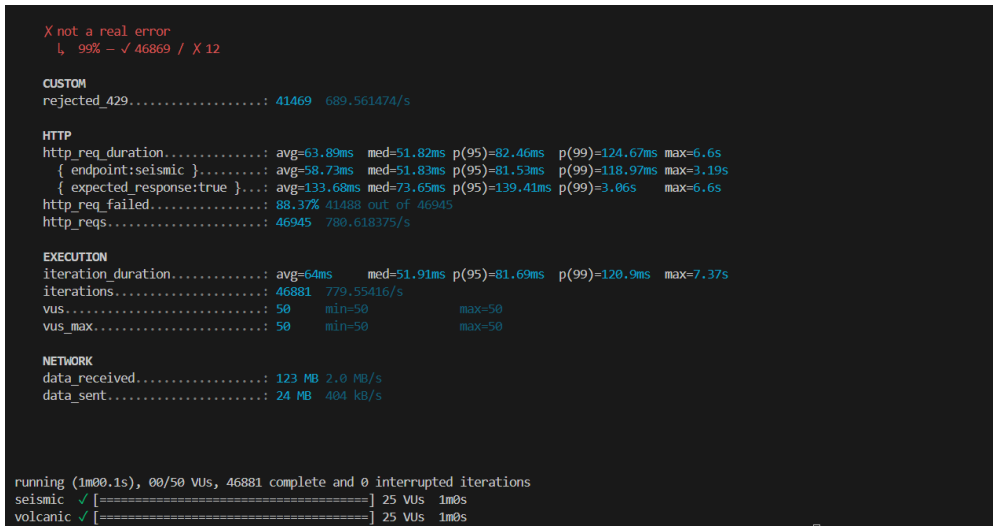
```
set -a
source <(tr -d '\r' < .env)
set +a
```

Command menjalankan load testing

```
k6 run --summary-export=result.json load_test.js
```

Hasil Load Testing





2.2.4.2. Penanganan skema Outage PVMBG

Pengujian dilakukan pada terminal wsl dengan langkah berikut:

1) Muat (*load)* env

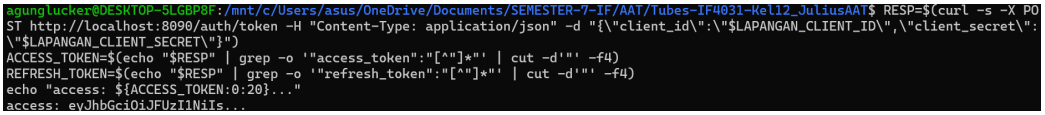
```
set -a
source <(tr -d '\r' < .env)
set +a
```

2) Melakukan Login ke dalam sistem di awal untuk men-*generate* access token dan refresh token

```
RESP=$(curl -s -X POST http://localhost:8090/auth/token
-H "Content-Type: application/json" -d
"{\"client_id\":\"$LAPANGAN_CLIENT_ID\",\"client_secret\"
:\"$LAPANGAN_CLIENT_SECRET\"}")

ACCESS_TOKEN=$(echo "$RESP" | grep -o
'"access_token":"[^"]*"' | cut -d'"' -f4)
REFRESH_TOKEN=$(echo "$RESP" | grep -o
'"refresh_token":"[^"]*"' | cut -d'"' -f4)

echo "access: ${ACCESS_TOKEN:0:20}..."
```



3) Menyalakan outage PVMBG melalui POST /admin/outage

```
curl -i -X POST http://localhost:8082/admin/outage -H
"Authorization: Bearer $PVMBG_TOKEN" \
  -H "Content-Type: application/json" -d '{"outage":
true}'
```

```
agunglucker@DESKTOP-5LGBP8F:/mnt/c/Users/asus/OneDrive/Documents/SEMESTER-7-IF/AAT/Tubes-IF4031-Kel12_JuliusAAT$ curl -i -X POST http
://localhost:8082/admin/outage \http://localhost:8082/admin/outage \
  -H "Authorization: Bearer $PVMBG_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"outage": true}'
HTTP/1.1 200 OK
X-Powered-By: Express
X-Correlation-ID: 9677f615-10f2-4620-a602-fbefed3aca81
Content-Type: application/json; charset=utf-8
Content-Length: 15
ETag: W/"f-VaSQ4oDUiZblZNAEkkN+sX+q3Sg"
Date: Thu, 08 Oct 2026 17:12:06 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"status":"ok"}HTTP/1.1 200 OK
X-Powered-By: Express
X-Correlation-ID: 8b3f8f1d-65fc-45f0-867b-6dbd63801e40
Content-Type: application/json; charset=utf-8
Content-Length: 15
ETag: W/"f-VaSQ4oDUiZblZNAEkkN+sX+q3Sg"
Date: Thu, 08 Oct 2026 17:12:09 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"status":"ok"}agunglucker@DESKTOP-5LGBP8F:/mnt/c/Users/asus/OneDrive/Documents/SEMESTER-7-IF/AAT/Tubes-IF4031-Kel12_JuliusAAT$
```

4) Melakukan percobaan permintaan data seismik ketika PVMBG sedang outage, bersamaan dengan melakukan refresh token agar token tidak expired

```
RESP=$(curl -s -X POST http://localhost:8090/auth/refresh
-H "Content-Type: application/json" -d
"{\"refresh_token\":\"$REFRESH_TOKEN\"}")
ACCESS_TOKEN=$(echo "$RESP" | grep -o
'"access_token":"[^"]*"' | cut -d'"' -f4)
REFRESH_TOKEN=$(echo "$RESP" | grep -o
'"refresh_token":"[^"]*"' | cut -d'"' -f4)


curl -i "http://localhost:8080/data?hazard_type=SEISMIC"
-H "Authorization: Bearer $ACCESS_TOKEN"
```

Screenshot Query dan potongan atas hasil data seismik


```
{"status":"ok"}agunglucker@DESKTOP-5LGBP8F:/mnt/c/Users/asus/OneDrive/Documents/SEMESTER-7-IF/AAT/Tubes-IF4031-KeRESP=$(curl -s -X PO
ST http://localhost:8090/auth/refresh -H "Content-Type: application/json" -d "{\"refresh_token\":\"$REFRESH_TOKEN\"}")RESH_TOKEN\"}")
ACCESS_TOKEN=$(echo "$RESP" | grep -o '"access_token":"[^"]*"' | cut -d'"' -f4)
REFRESH_TOKEN=$(echo "$RESP" | grep -o '"refresh_token":"[^"]*"' | cut -d'"' -f4)

curl -i "http://localhost:8080/data?hazard_type=SEISMIC" -H "Authorization: Bearer $ACCESS_TOKEN"
HTTP/1.1 200 OK
X-Correlation-ID: b3aea149-555a-4e61-9872-09e22fe452b6
Cache-Control: no-store
Content-Type: application/json; charset=utf-8
Content-Length: 21340
ETag: W/"535c-0SVxZpU0qfuekGCioXKkfDrdwc0"
Date: Thu, 08 Oct 2026 17:13:14 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"data":[{"hazard_id":"BMKG-SE001","source":"BMKG","hazard_type":"SEISMIC","severity":"NORMAL","area_name":"Selat Sunda","occurred_at
":"2026-09-09T19:15:00.000Z","ingested_at":"2026-10-08T17:08:17.700Z","source_ref_id":"SE001","latitude":-6.21,"longitude":106.82,"at
tributes":{"depth_km":32.5,"magnitude":4.8,"potential_tsunami":false}},{"hazard_id":"BMKG-SE002","source":"BMKG","hazard_type":"SEISM
IC","severity":"WASPADA","area_name":"Bali","occurred_at":"2026-09-10T07:42:00.000Z","ingested_at":"2026-10-08T17:08:17.730Z","source
_ref_id":"SE002","latitude":-8.32,"longitude":115.41,"attributes":{"depth_km":18.7,"magnitude":5.2,"tsunami_warning":{"warning_id":"T
W001","threat_level":"Waspada","affected_zones":["Pantai Selatan Bali","Nusa Penida"],"related_event_id":"SE002","estimated_arrival":
"2026-09-10T08:30:00.000Z"},"potential_tsunami":true}},{"hazard_id":"BMKG-SE003","source":"BMKG","hazard_type":"SEISMIC","severity":"
NORMAL","area_name":"Sumatera Barat","occurred_at":"2026-09-10T23:30:00.000Z","ingested_at":"2026-10-08T17:08:17.734Z","source_ref_id
":"SE003","latitude":-0.91,"longitude":100.46,"attributes":{"depth_km":76.2,"magnitude":4.5,"potential_tsunami":false}},{"hazard_id":
"BMKG-SE004","source":"BMKG","hazard_type":"SEISMIC","severity":"SIAGA","area_name":"Ambon, Maluku","occurred_at":"2026-09-11T12:05:0
0.000Z","ingested_at":"2026-10-08T17:08:17.737Z","source_ref_id":"SE004","latitude":-3.72,"longitude":128.18,"attributes":{"depth_km"
:24.8,"magnitude":6.1,"tsunami_warning":{"warning_id":"TW002","threat_level":"Siaga","affected_zones":["Ambon","Seram Bagian Barat"],
"related_event_id":"SE004","estimated_arrival":"2026-09-11T13:10:00.000Z"},"potential_tsunami":true}},{"hazard_id":"BMKG-SE005","sour
ce":"BMKG","hazard_type":"SEISMIC","severity":"NORMAL","area_name":"Jawa Tengah","occurred_at":"2026-09-11T21:17:00.000Z","ingested_a
t":"2026-10-08T17:08:17.749Z","source_ref_id":"SE005","latitude":-7.81,"longitude":110.37,"attributes":{"depth_km":91.4,"magnitude":4
.2,"potential_tsunami":false}},{"hazard_id":"BMKG-SE006","source":"BMKG","hazard_type":"SEISMIC","severity":"WASPADA","area_name":"Su
mba, Nusa Tenggara Timur","occurred_at":"2026-09-12T09:23:00.000Z","ingested_at":"2026-10-08T17:08:17.764Z","source_ref_id":"SE006","
latitude":-9.12,"longitude":119.48,"attributes":{"depth_km":42.1,"magnitude":5.7,"tsunami_warning":{"warning_id":"TW003","threat_leve
l":"Waspada","affected_zones":["Waingapu","Sumba Timur"],"related_event_id":"SE006","estimated_arrival":"2026-09-12T10:40:00.000Z"},"
potential_tsunami":true}},{"hazard_id":"BMKG-SE007","source":"BMKG","hazard_type":"SEISMIC","severity":"WASPADA","area_name":"Bengkul
u","occurred_at":"2026-09-13T01:11:00.000Z","ingested_at":"2026-10-08T17:08:17.773Z","source_ref_id":"SE007","latitude":-2.54,"longit
```

Screenshot Potongan bawah hasil data seismik

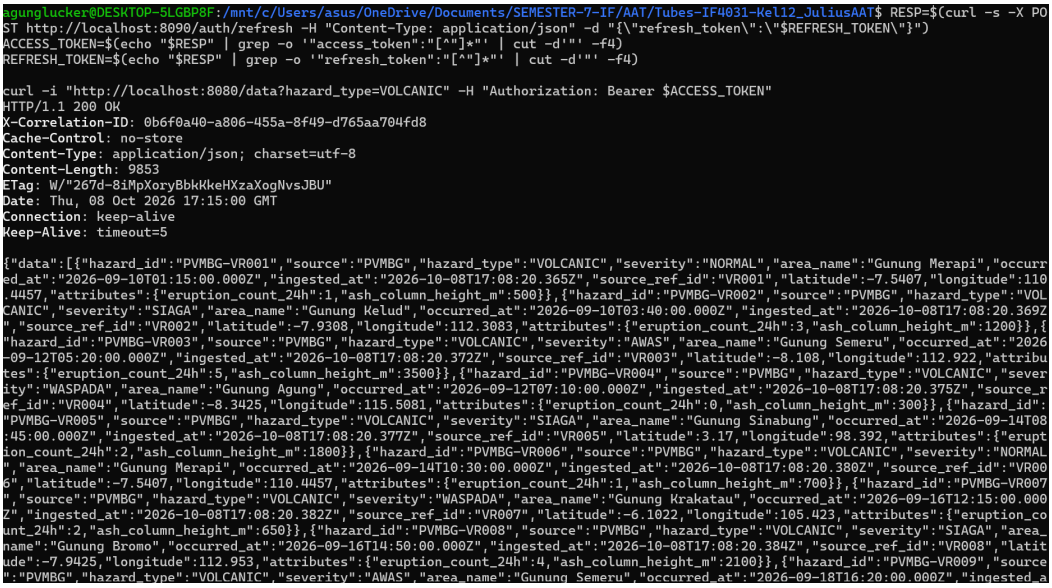


5) Melakukan percobaan permintaan data Vulkanik ketika PVMBG sedang outage, bersamaan dengan refresh token, dengan expected hasil data vulkanik terakhir dari canonical store beserta penanda stale_since

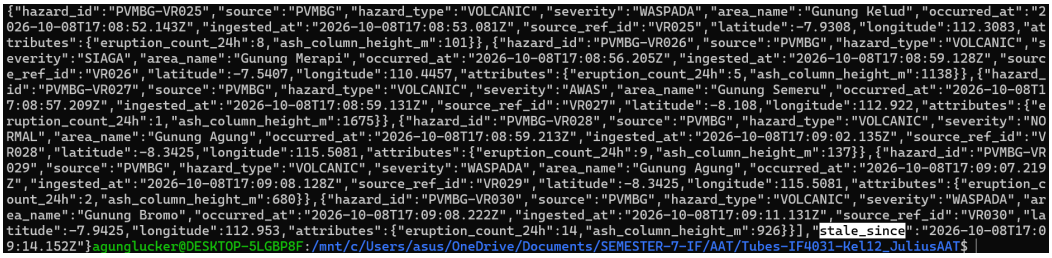
```
RESP=$(curl -s -X POST http://localhost:8090/auth/refresh
-H "Content-Type: application/json" -d
"{\"refresh_token\":\"$REFRESH_TOKEN\"}")
ACCESS_TOKEN=$(echo "$RESP" | grep -o
'"access_token":"[^"]*"' | cut -d'"' -f4)
REFRESH_TOKEN=$(echo "$RESP" | grep -o
'"refresh_token":"[^"]*"' | cut -d'"' -f4)


curl -i "http://localhost:8080/data?hazard_type=VOLCANIC"
-H "Authorization: Bearer $ACCESS_TOKEN"
```

Screenshot Query dan potongan atas hasil data vulkanik



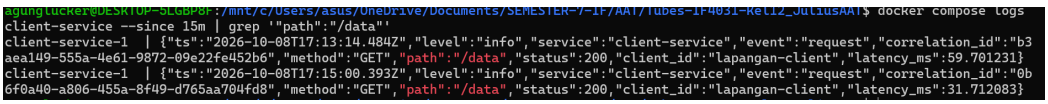
Screenshot potongan bawah hasil data vulkanik



Terdapat penanda *stale_since* pada hasil data vulkanik yang diambil dari canonical store (via aggregator ke client-service) ketika PVMBG outage.

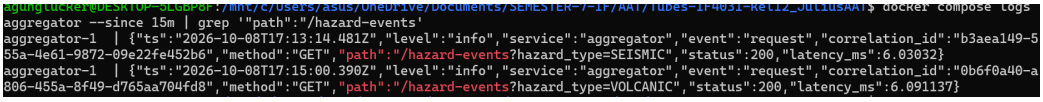
6) Menampilkan tampilan logs dari client-service yang difilter untuk logging permintaan data (seismik dan vulkanik) dengan correlation id nya

```
docker compose logs client-service --since 15m | grep
'"path":"/data"'
```



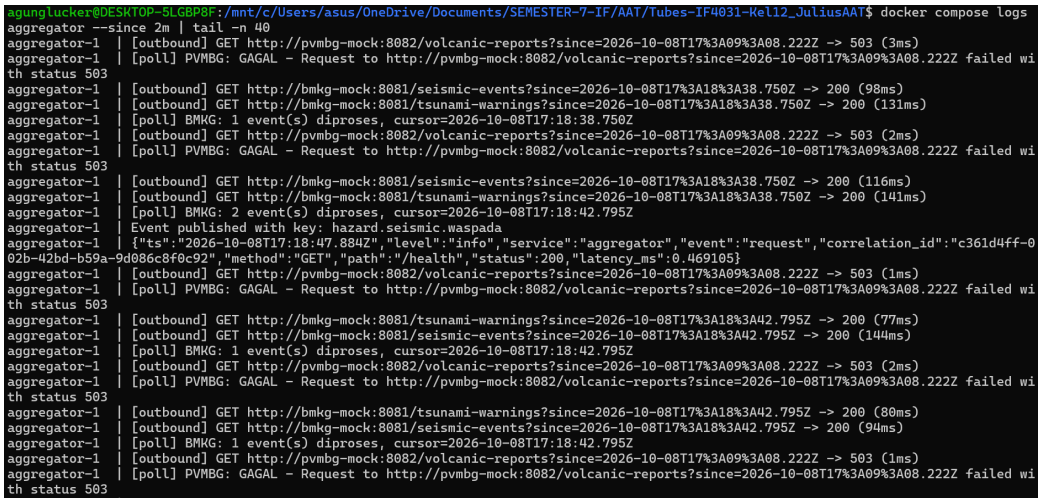
7) Menampilkan tampilan logs dari aggregator service yang difilter untuk logging dua pengambilan data sebelumnya (seismik dan vulkanik) dengan correlation id nya yang diteruskan dari client-service

```
docker compose logs aggregator --since 15m | grep
'"path":"/hazard-events'
```



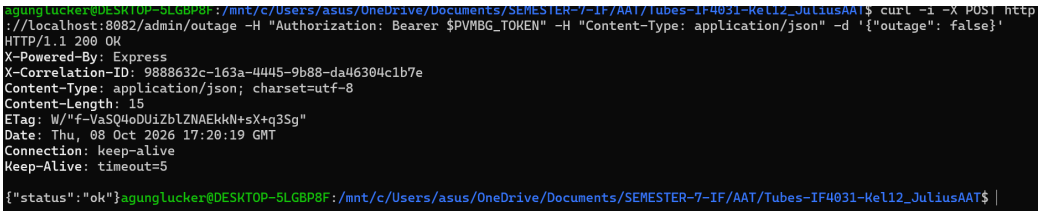
8) Menampilkan logs terakhir dari aggregator service (tanpa filter) selama PVMBG mengalami outage.

```
docker compose logs aggregator --since 2m | tail -n 40
```



9) Mematikan outage pada PVMBG mock

```
curl -i -X POST http://localhost:8082/admin/outage -H
"Authorization: Bearer $PVMBG_TOKEN" -H "Content-Type:
application/json" -d '{"outage": false}'
HTTP/1.1 200 OK
```

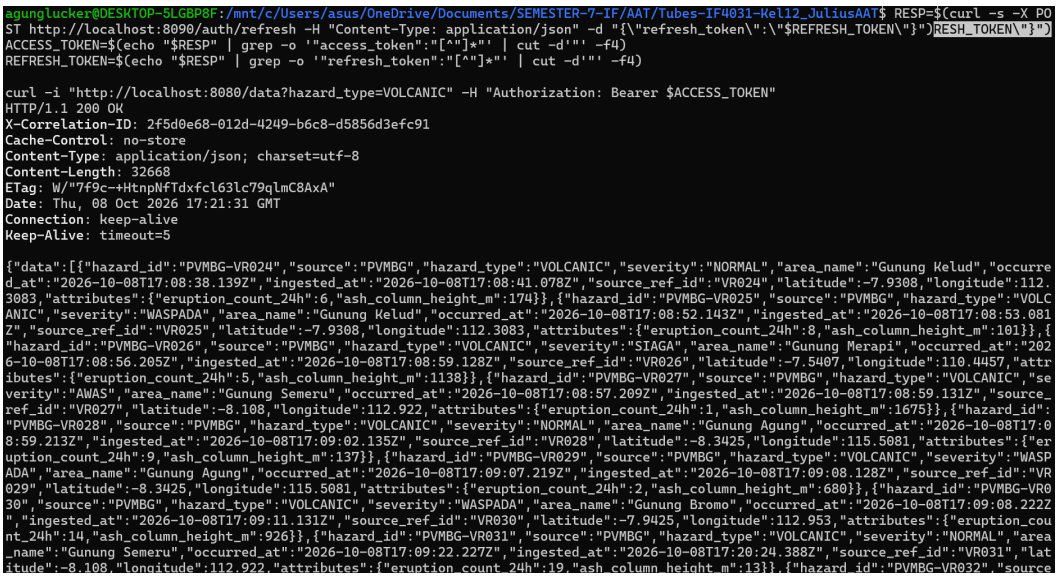


10) Menampilan bukti permintaan data vulkanik yang normal (tanpa stale_since) setelah outage dimatikan pada PVMBG

```
RESP=$(curl -s -X POST http://localhost:8090/auth/refresh
-H "Content-Type: application/json" -d
"{\"refresh_token\":\"$REFRESH_TOKEN\"}")
ACCESS_TOKEN=$(echo "$RESP" | grep -o
'"access_token":"[^"]*"' | cut -d'"' -f4)
REFRESH_TOKEN=$(echo "$RESP" | grep -o
'"refresh_token":"[^"]*"' | cut -d'"' -f4)

curl -i "http://localhost:8080/data?hazard_type=VOLCANIC"
-H "Authorization: Bearer $ACCESS_TOKEN"
```

Screenshot query dan potongan atas hasil data vulkanik



Screenshot potongan bawah hasil data vulkanik



Hasil pengambilan data vulkanik kembali normal, dengan tidak adanya field stale_since, yang menandakan bahwa tidak ada outage pada PVMBG.

2.2.5. Jawaban atas butir "Di laporan"

Model konkurensi yang dipilih adalah pemisahan proses ingest dan proses query melalui polling berkala dan Canonical Store, dilengkapi dengan pemanggilan paralel ke BMKG dan PVMBG pada setiap siklus polling, rate limiting pada endpoint query, dan pencatatan status tiap sumber secara independen.

Model ini mencegah head-of-line blocking karena permintaan client tidak pernah menunggu respons langsung dari BMKG maupun PVMBG. Model ini mencegah resource exhaustion karena jumlah permintaan yang diproses dan jumlah koneksi database dibatasi secara eksplisit, sehingga lonjakan permintaan direspons dengan penolakan terkendali, bukan kehabisan resource. Model ini mendukung graceful degradation karena status tiap sumber dicatat secara independen, sehingga kegagalan satu sumber tidak menghentikan pelayanan data dari sumber lain.

Load test dilakukan menggunakan k6 melalui skrip load_test.js. Pengujian menjalankan 50 virtual user (VU) secara paralel selama 60 detik dengan executor constant-vus. Setiap VU mengirim permintaan berikutnya segera setelah respons sebelumnya diterima, tanpa think time. Sebanyak 25 VU meminta data seismik (BMKG) dan 25 VU lainnya meminta data vulkanik (PVMBG) ke endpoint GET /data pada Aggregator BNPB. Selama pengujian, delay PVMBG diatur 3 detik untuk mengamati apakah permintaan BMKG-only tertahan oleh permintaan PVMBG yang lambat. Sebelum mengirim permintaan pertamanya, setiap VU melakukan satu kali login ke bnpb-token-service sebagai client downstream ber-scope Media, dengan kredensial yang dibaca dari environment variable. Setiap VU menyimpan token miliknya sendiri. Apabila suatu permintaan menerima respons 401 karena access token kadaluarsa, VU tersebut memperbarui token melalui endpoint refresh (atau login ulang jika refresh gagal) lalu mengulang permintaannya satu kali. Metrik yang dilaporkan adalah throughput median (p50), p95, p99, dan nilai maksimum latensi dan error rate.

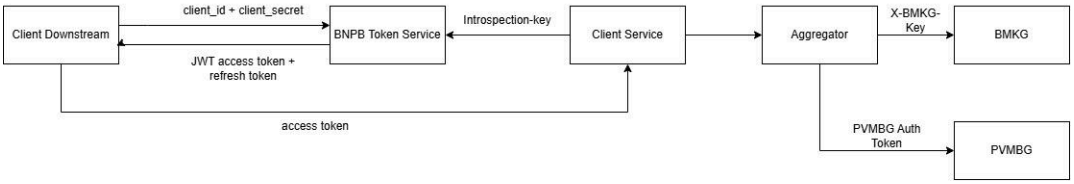
2.3. Service-to-Service Authentication & Trust

2.3.1. Rancangan Solusi

Platform menghubungkan dua instansi sumber yang memiliki domain kepercayaan berbeda, yaitu BMKG dan PVMBG. Kredensial yang berlaku pada satu instansi tidak boleh digunakan untuk mengakses instansi lain. Selain itu, client downstream tidak boleh menggunakan kredensial service

sumber untuk memperoleh data BNPB, tetapi hanya melalui Client Service.

Oleh karena itu, terdapat 3 jenis kredensial pada desain arsitektur kami, yaitu kredensial Aggregator ke BMKG, Aggregator ke PVMBG, dan client downstream ke BNPB.



BMKG menggunakan API key yang divalidasi oleh BMKG. Aggregator mengambil nilai tersebut dari environment variable dan mengirimnya melalui X-BMKG-KEY. PVMBG menggunakan credential terpisah dengan format Authorization. Aggregator mengambil nilai tersebut dari environment variable. Dengan demikian, kredensial BMKG tidak dapat digunakan untuk mengakses PVMBG dan sebaliknya.

Autentikasi client downstream:

Client	Environment variable	Scope	Hak data
Media / Pers	MEDIA_CLIENT_ID, MEDIA_CLIENT_SECRET	summary	Field ringkasan
Tim Lapangan	LAPANGAN_CLIENT_ID, LAPANGAN_CLIENT_SECRET	full	Field ringkasan dan mentah
BNPB Internal Ops	INTERNAL_CLIENT_ID, INTERNAL_CLIENT_SECRET	full	Field ringkasan dan mentah

Token Service menggunakan tiga identitas yang berbeda untuk setiap jenis client downstream (Media, Lapangan, dan Internal). Scope setiap jenis client downstream dicatat pada basis data Token Service. Akibatnya, client tidak dapat mengubah scope-nya sendiri.

2.3.1.1. Access token dan refresh token

Access token merupakan JWT yang ditandatangani menggunakan ES256. Payload token terdiri dari sub (identitas client), scope, sid (identitas session), dan jti. Client service melakukan verifikasi pada access token JWT menggunakan public key. Kemudian, Client Service memeriksa keaktifan token tersebut dengan memanggil endpoint introspect Token Service.

Refresh token disimpan pada basis data Token Service. Token ini hanya dapat digunakan sekali saja, sehingga terdapat field *used* pada basis data tersebut.

2.3.1.2. Pembatasan data berdasarkan scope

Pembatasan field dilakukan oleh client service. Client downstream yang meminta field diluar scope akan menerima HTTP 403, dengan error insuficient_scope.

2.3.2. Alasan keputusan

Alternatif	Kelebihan	Kekurangan
API key untuk semua service	Implementasi sederhana	Tidak dapat memodelkan scope, expiry, dan refresh dengan baik
JWT tanpa introspection	Client Service tidak perlu memanggil Token Service setiap request	Token yang sudah dicabut tetap valid sampai expired
JWT + introspection (dipilih)	Mendukung expiry sekaligus pencabutan session	Menambah dependency runtime ke Token Service
OAuth2/OIDC penuh	Standar dan fitur lengkap	Terlalu kompleks untuk diimplementasikan dan kebutuhan milestone ini.

2.3.3. Batasan

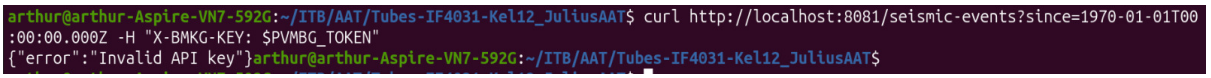
Batasan yang disadari pada implementasi autentikasi adalah sistem belum mencakup mekanisme rotasi kunci secara dinamis/ otomatis. Selain itu, endpoint login dan refresh belum memiliki rate limiting khusus sehingga masih berpotensi menerima percobaan request berulang dalam jumlah besar. Credential dan secret masih dikelola melalui environment variable, sehingga pada deployment produksi

sebaiknya dipindahkan ke secret manager. Client Service juga masih bergantung pada Token Service untuk melakukan introspection sesi, sehingga apabila Token Service tidak tersedia, validasi sesi dapat terganggu meskipun token JWT secara kriptografis masih valid.

2.3.4. Bukti Penyelesaian

2.3.4.1. Mengakses BMKG dan PVMBG

Token PVMBG juga tidak dapat digunakan untuk mengakses BMKG.

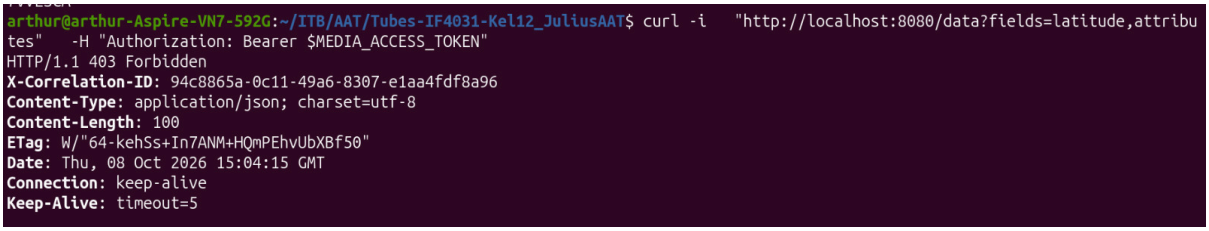


API key BMKG tidak dapat digunakan untuk mengakses PVMBG.

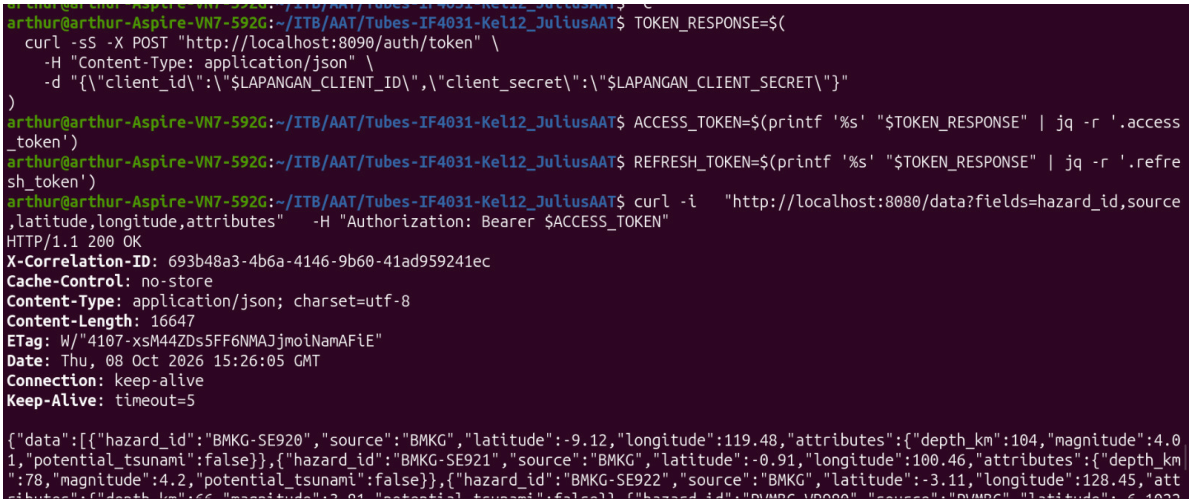


2.3.4.2. Media Meminta Data Mentah

Akses data yang masuk kategori mentah oleh media akan ditolak.



2.3.4.3. Lakukan Refresh Token



Setelah 60 detik, access token akan expired.

```
:false}}]}arthur@arthur-Aspire-VN7-592G:~/ITB/AAT/Tubes-IF4031-Kel12_JuliusAAT$ curl -i   "http://localhost:8080/data?fields=hazard
_id,source,latitude,longitude,attributes"   -H "Authorization: Bearer $ACCESS_TOKEN"
HTTP/1.1 401 Unauthorized
X-Correlation-ID: 0cd81460-d178-4bc9-a79c-f6a7ad7863a5
Content-Type: application/json; charset=utf-8
Content-Length: 61
ETag: W/"3d-MiarpMCJXsh5Um2+vaKKjrVJfRk"
Date: Thu, 08 Oct 2026 15:27:11 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"invalid_token","error_description":"Invalid Token"}arthur@arthur-Aspire-VN7-592G:~/ITB/AAT/Tubes-IF4031-Kel12_JuliusAAT$
```

Lakukan refresh untuk mendapatkan access token yang baru.
Token tersebut dapat digunakan untuk mengambil data kembali.

```
arthur@arthur-Aspire-VN7-592G:~/ITB/AAT/Tubes-IF4031-Kel12_JuliusAAT$ REFRESH_RESPONSE=$(
  curl -sS -X POST "http://localhost:8090/auth/refresh" \
    -H "Content-Type: application/json" \
    -d "{\"refresh_token\":\"$REFRESH_TOKEN\"}"
)
arthur@arthur-Aspire-VN7-592G:~/ITB/AAT/Tubes-IF4031-Kel12_JuliusAAT$ NEW_ACCESS_TOKEN=$(printf '%s' "$REFRESH_RESPONSE" | jq -r '.
access_token')
NEW_REFRESH_TOKEN=$(printf '%s' "$REFRESH_RESPONSE" | jq -r '.refresh_token')
arthur@arthur-Aspire-VN7-592G:~/ITB/AAT/Tubes-IF4031-Kel12_JuliusAAT$ curl -i \
  "http://localhost:8080/data?fields=hazard_id,source,latitude,longitude,attributes" \
  -H "Authorization: Bearer $NEW_ACCESS_TOKEN"
HTTP/1.1 200 OK
X-Correlation-ID: 087619a4-fa7d-4581-8fbe-6073243c3989
Cache-Control: no-store
Content-Type: application/json; charset=utf-8
Content-Length: 16647
ETag: W/"4107-xsM44ZDs5FF6NMAJjmoiNamAFiE"
Date: Thu, 08 Oct 2026 15:30:46 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"data":[{"hazard_id":"BMKG-SE920","source":"BMKG","latitude":-9.12,"longitude":119.48,"attributes":{"depth_km":104,"magnitude":4.0
1,"potential_tsunami":false}},{"hazard_id":"BMKG-SE921","source":"BMKG","latitude":-0.91,"longitude":100.46,"attributes":{"depth_km
":78,"magnitude":4.2,"potential_tsunami":false}},{"hazard_id":"BMKG-SE922","source":"BMKG","latitude":-3.11,"longitude":128.45,"att
```

Akses token lama menjadi tidak dapat digunakan mengakses data.

```
:false}}]}arthur@arthur-Aspire-VN7-592G:~/ITB/AAT/Tubes-IF4031-Kel12_JuliusAAT$ curl -i   "http://localhost:8080/data?fields=hazard
_id,source,latitude,longitude,attributes"   -H "Authorization: Bearer $ACCESS_TOKEN"
HTTP/1.1 401 Unauthorized
X-Correlation-ID: 584024da-d661-4f5f-85cd-71f8a36b23d4
Content-Type: application/json; charset=utf-8
Content-Length: 61
ETag: W/"3d-MiarpMCJXsh5Um2+vaKKjrVJfRk"
Date: Thu, 08 Oct 2026 15:32:00 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"invalid_token","error_description":"Invalid Token"}arthur@arthur-Aspire-VN7-592G:~/ITB/AAT/Tubes-IF4031-Kel12_JuliusAAT$
```

2.3.5. Jawaban atas butir "Di laporan"

Mekanisme yang dipakai adalah pemisahan credential per instansi, JWT untuk client downstream, scope server-side untuk pembatasan field, introspection untuk validasi sesi, dan refresh token one-time yang disimpan sebagai hash.

Mekanisme tersebut dipilih karena API key BMKG dan token PVMBG tidak dapat dipertukarkan, sehingga kompromi satu instansi tidak langsung membuka instansi lain. JWT diverifikasi

menggunakan public key, sedangkan private key dan secret dibaca dari environment atau file konfigurasi yang tidak dibagikan kepada client. Scope diputuskan oleh Token Service dan diverifikasi Client Service, sehingga client tidak dapat memperluas hak akses hanya dengan mengubah parameter request dan data mentah tetap difilter di sisi server (Client Service), bukan hanya diandalkan pada sisi client

Risiko yang disadari adalah ketergantungan introspection terhadap Token Service dan kebutuhan secret manager pada deployment produksi. Namun, risiko tersebut tidak mengubah pemisahan trust domain yang menjadi tujuan Problem 3.

2.4. Arsitektur Microservice & Flexible Storage

2.4.1. Rancangan Solusi

Permasalahan yang diangkat adalah seluruh logika sistem, mulai dari polling data, pemetaan skema, autentikasi, hingga penyajian data, berada dalam satu unit *deployment*. Akibatnya, perubahan pada salah satu komponen berpotensi mengharuskan seluruh sistem untuk di-*build* dan di-*deploy* ulang. Untuk mengatasi permasalahan tersebut, sistem dipecah menjadi beberapa *service* yang independen. Dengan pendekatan ini, setiap *service* dapat di-*build*, dihentikan, dan dijalankan ulang secara terpisah tanpa mengganggu operasional *service* lainnya.

Pada *service* Aggregator, penyimpanan data kanonik menggunakan format JSON dengan kebijakan *additive-only*, sebagaimana diterapkan pada Problem 1. Melalui kebijakan ini, struktur utama data kanonik tetap dipertahankan, sedangkan field tambahan yang tidak termasuk dalam skema kanonik diteruskan ke dalam objek attributes. Dengan demikian, sistem dapat mengakomodasi penambahan field dari sumber data tanpa mengubah struktur utama HazardEvent atau memerlukan perubahan kode pemetaan setiap kali skema pada *service* sumber mengalami penambahan field.

Sementara itu, untuk penyimpanan data pada masing-masing *service*, kami menerapkan pembatasan akses basis data berdasarkan kepemilikan *service*. Setiap *service* dikonfigurasi menggunakan environment variable yang sesuai dengan kredensial dan basis data miliknya. Selain itu, port basis data tidak diekspos ke host, sehingga akses langsung dari luar

jaringan internal container dibatasi. Kami juga menerapkan jaringan internal untuk mengatur komunikasi antar komponen, sehingga setiap *service* hanya dapat berkomunikasi dengan komponen yang memang diperlukan sesuai dengan arsitektur sistem. Pendekatan ini bertujuan untuk membatasi akses ke basis data, mengurangi permukaan serangan, dan menjaga isolasi antar-*service*.

2.4.2. Alasan keputusan

Keputusan : Penyimpanan untuk service

Alternatif	Kelebihan	Kekurangan
MongoDB	Schemaless native, sehingga field baru langsung tersimpan tanpa migrasi	Tidak ada *sequence* native untuk ID berurutan otomatis, sehingga perlu koleksi *counter* dengan $inc. Constraint di level DB lebih longgar.
PostgreSQL(dipilih)	Memiliki constraint yang ketat sehingga increment id khususnya pada service PVMBG dan BMKG bisa digenerate secara otomatis. Selain itu postgres menyediakan JSONB yang memberikan kebebasan untuk menambah field secara dinamis dan dapat menerapkan Indeks untuk mempercepat query jika dibutuhkan dimasa mendatang	Isi JSONB tidak divalidasi di level DB. Query ke field dalam JSONB lebih verbose dan butuh indeks agar cepat. Perubahan pada kolom kanonik tetap memerlukan migrasi.
Key-value store	Baca dan tulis sangat cepat, schemaless, dan dapat menyimpan penanda data terakhir.	Query berdasarkan waktu atau sumber tidak native, sehingga indeks (mis. sorted set) harus dibangun dan dijaga manual.

2.4.3. Batasan

1. Tidak ada replikasi pada Canonical Store. Bila service store mati, Aggregator tidak dapat menulis maupun melayani data.
2. Aggregator tidak memvalidasi isi event dari sumber. Sumber (BMKG/PVMBG) diasumsikan mengirim data yang sesuai

kontrak. Mapper hanya memetakan field dan meneruskan field tambahan ke attributes.

2.4.4. Bukti Penyelesaian

2.4.4.1. Setiap komponen berjalan sebagai container terpisah dan dapat di-rebuild/dijalankan ulang tanpa rebuild/restart container lain

1) Buat dua terminal wsl, terminal 1 untuk menampilkan daftar kontainer pada docker beserta status nya, sedangkan terminal 2 untuk menjalankan loop pemantauan pada sistem, dengan sample nya mengirim request ke endpoint GET /data?hazard_type=SEISMIC milik Client Service setiap detik

2) Pada Terminal 2, lakukan login pada sistem untuk mendapatkan access token

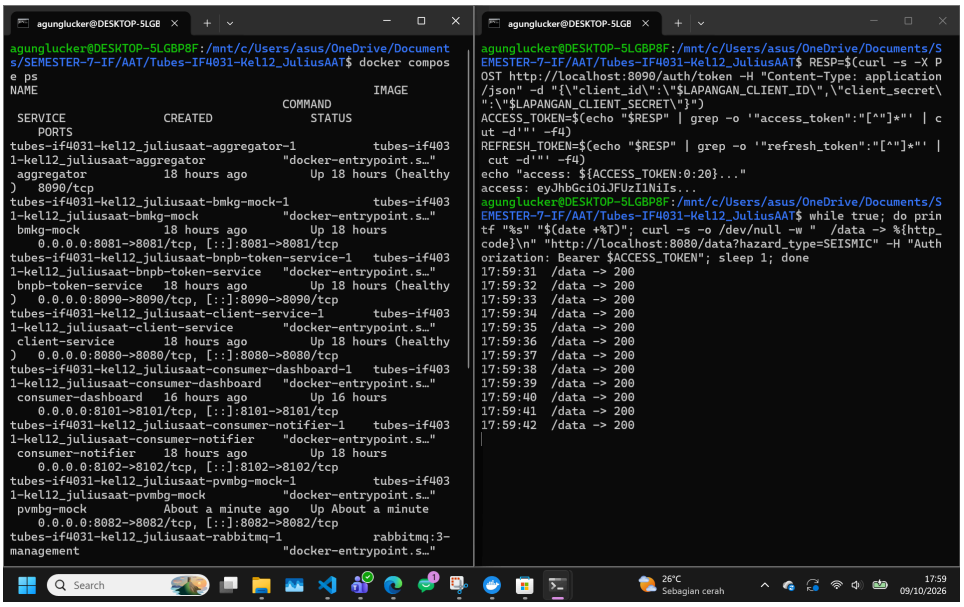
```
RESP=$(curl -s -X POST
http://localhost:8090/auth/token -H "Content-Type:
application/json" -d
"{\"client_id\":\"$LAPANGAN_CLIENT_ID\",\"client_s
ecret\":\"$LAPANGAN_CLIENT_SECRET\"}")
ACCESS_TOKEN=$(echo "$RESP" | grep -o
'"access_token":"[^"]*"' | cut -d'"' -f4)
REFRESH_TOKEN=$(echo "$RESP" | grep -o
'"refresh_token":"[^"]*"' | cut -d'"' -f4)
echo "access: ${ACCESS_TOKEN:0:20}..."
access: eyJhbGciOiJFUzI1NiIs...
```

3) Selanjutnya, Terminal 2 menjalankan loop pemantauan dari request ke endpoint GET /data?hazard_type=SEISMIC(mengambil data seismik) setiap detik dengan access token yang didapatkan sebelumnya

```
while true; do printf "%s" "$(date +%T)"; curl -s
-o /dev/null -w "  /data -> %{http_code}\n"
"http://localhost:8080/data?hazard_type=SEISMIC"
-H "Authorization: Bearer $ACCESS_TOKEN"; sleep 1;
done
```

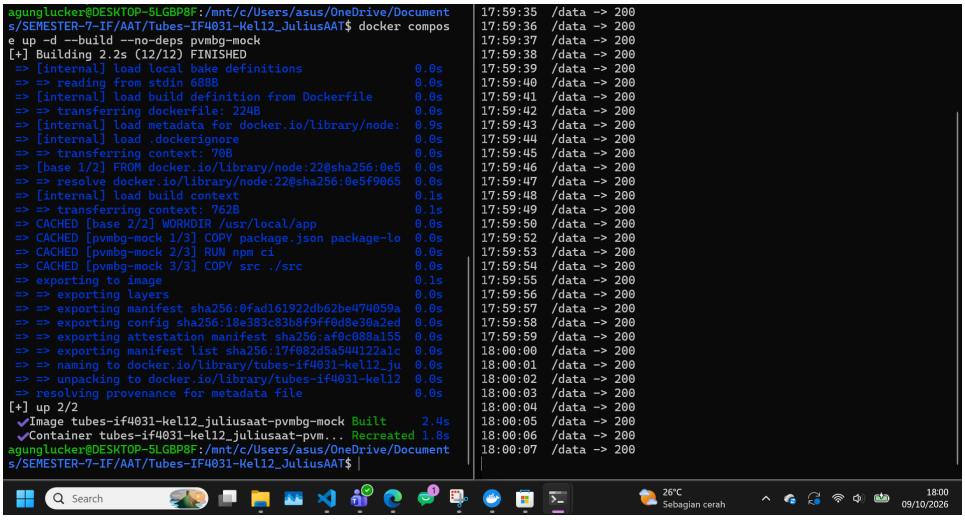
4) Bersamaan dengan itu, Terminal 1 menampilkan status dari kontainer-kontainer docker pada sistem

```
Docker Compose ps
```



5) Pada terminal 1, dilakukan rebuild ulang pada service Pvmbg-mock sembari menampilkan loop pemantauan request data seismik di terminal 2

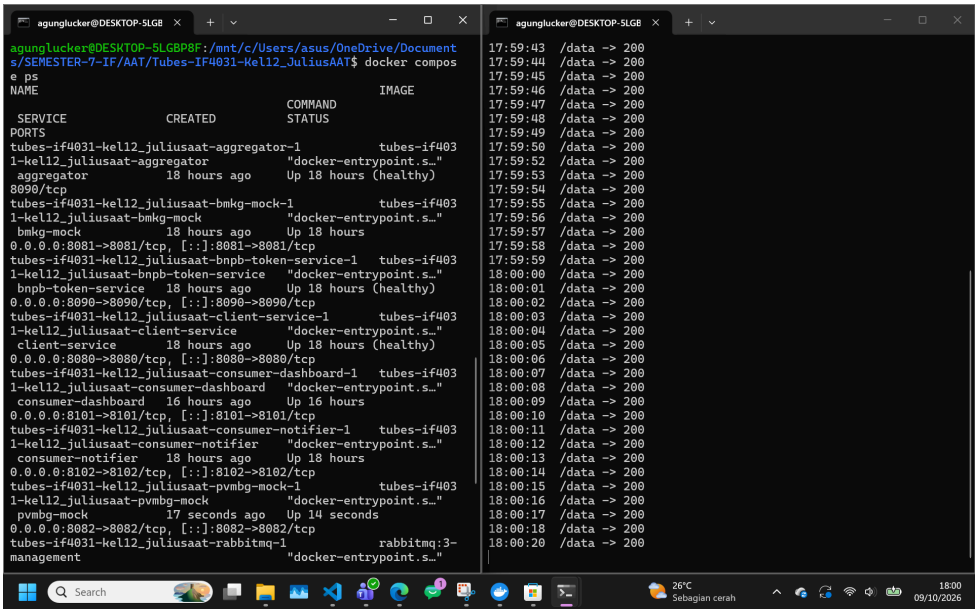
```
docker compose up -d --build --no-deps pvmbg-mock
```



Request data seismik pada client-service tetap berjalan walaupun PVMBG-mock service sedang di-rebuild

6) Setelah rebuild service pvmbg-mock selesai, tampilkan kembali daftar kontainer docker beserta status dari sistem bersamaan dengan loop pemantauan request data seismik nya

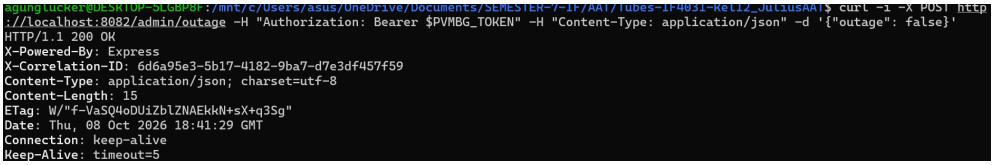
```
docker compose ps
```



2.4.4.2. Record dari sebelum dan sesudah perubahan skema tersimpan berdampingan dan keduanya dapat dibaca

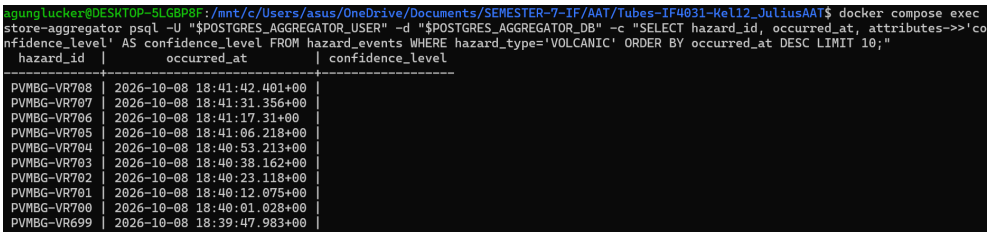
1) Pastikan Outage pada PVMBG telah dimatikan sebelum melakukan perubahan skema pada PVMBG (opsional)

```
curl -i -X POST http://localhost:8082/admin/outage -H "Authorization: Bearer $PVMBG_TOKEN" -H "Content-Type: application/json" -d '{"outage": false}'
```



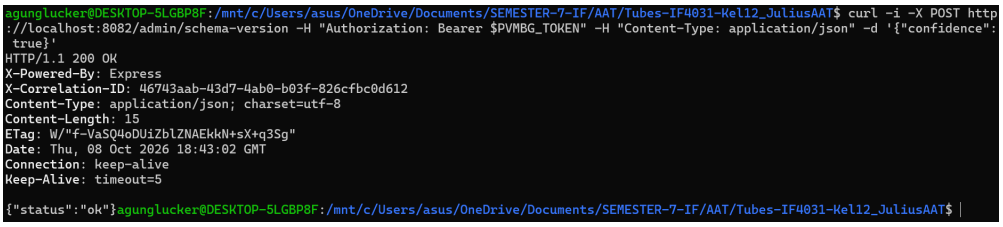
2) Menampilkan kondisi record pada Store Aggregator untuk data vulkanik sebelum perubahan skema pada data VolcanicReport

```
docker compose exec store-aggregator psql -U
"$POSTGRES_AGGREGATOR_USER" -d
"$POSTGRES_AGGREGATOR_DB" -c "SELECT hazard_id,
occurred_at, attributes->>'confidence_level' AS
confidence_level FROM hazard_events WHERE
hazard_type='VOLCANIC' ORDER BY occurred_at DESC
LIMIT 10;"
```



3) Melakukan perubahan skema via endpoint POST /admin/schema-version saat sistem berjalan

```
curl -i -X POST
http://localhost:8082/admin/schema-version -H
"Authorization: Bearer $PVMBG_TOKEN" -H
"Content-Type: application/json" -d
'{"confidence": true}'
```



4) Tunggu beberapa detik hingga terjadi siklus polling, kemudian menampilkan kondisi record pada Store Aggregator untuk data vulkanik setelah perubahan skema pada data VolcanicReport.

```
docker compose exec store-aggregator psql -U
"$POSTGRES_AGGREGATOR_USER" -d
"$POSTGRES_AGGREGATOR_DB" -c "SELECT hazard_id,
occurred_at, attributes->>'confidence_level' AS
confidence_level FROM hazard_events WHERE
```

```
hazard_type='VOLCANIC' ORDER BY occurred_at DESC
LIMIT 15;"
```

```
agunglucker@DESKTOP-5LGBP8F:/mnt/c/Users/asus/OneDrive/Documents/SEMESTER-7-IF/AAT/Tubes-IF4031-Kel12_JuliusAAT$ docker compose exec
store-aggregator psql -U "$POSTGRES_AGGREGATOR_USER" -d "$POSTGRES_AGGREGATOR_DB" -c "SELECT hazard_id, occurred_at, attributes->>'co
nfidence_level' AS confidence_level FROM hazard_events WHERE hazard_type='VOLCANIC' ORDER BY occurred_at DESC LIMIT 15;"
  hazard_id  |         occurred_at          | confidence_level
-------------+------------------------------+------------------
 PVMBG-VR722 | 2026-10-08 18:43:49.068+00 | 0.86
 PVMBG-VR721 | 2026-10-08 18:43:38.028+00 | 0.95
 PVMBG-VR720 | 2026-10-08 18:43:37.024+00 | 0.96
 PVMBG-VR719 | 2026-10-08 18:43:26.97+00  | 0.99
 PVMBG-VR718 | 2026-10-08 18:43:19.959+00 | 0.92
 PVMBG-VR717 | 2026-10-08 18:43:07.874+00 | 0.91
 PVMBG-VR716 | 2026-10-08 18:43:03.868+00 | 0.91
 PVMBG-VR715 | 2026-10-08 18:42:52.785+00 | 0.87
 PVMBG-VR714 | 2026-10-08 18:42:38.704+00 |
 PVMBG-VR713 | 2026-10-08 18:42:32.636+00 |
 PVMBG-VR712 | 2026-10-08 18:42:21.587+00 |
 PVMBG-VR711 | 2026-10-08 18:42:12.58+00  |
 PVMBG-VR710 | 2026-10-08 18:42:08.493+00 |
 PVMBG-VR709 | 2026-10-08 18:41:54.448+00 |
 PVMBG-VR708 | 2026-10-08 18:41:42.401+00 |
(15 rows)
```

2.4.4.3. Canonical Store hanya dapat diakses langsung oleh Aggregator, dan komponen lain memperoleh data melalui API Aggregator

1) Aggregator dapat mengakses Canonical Store

```
agunglucker@DESKTOP-5LGBP8F:/mnt/c/Users/asus/OneDrive/Documents/SEMESTER-7-IF/AAT/Tubes-IF4031-Kel12_JuliusAAT$ docker compose exec
aggregator sh -c 'getent hosts store-aggregator && echo "SUCCESS" || echo "FAIL"'
172.23.0.2      store-aggregator
SUCCESS
```

```
agunglucker@DESKTOP-5LGBP8F:/mnt/c/Users/asus/OneDrive/Documents/SEMESTER-7-IF/AAT/Tubes-IF4031-Kel12_JuliusAAT$ docker compose exec
aggregator sh -c 'timeout 3 bash -c "</dev/tcp/store-aggregator/5432" && echo SUCCESS || echo FAIL'
SUCCESS
```

2) Client Service tidak bisa mengakses Canonical Store

```
agunglucker@DESKTOP-5LGBP8F:/mnt/c/Users/asus/OneDrive/Documents/SEMESTER-7-IF/AAT/Tubes-IF4031-Kel12_JuliusAAT$ docker compose exec
client-service sh -c 'getent hosts store-aggregator && echo "SUCCESS" || echo "FAIL"'
FAIL
```

3) Client Service tetap dapat memperoleh data melalui API Aggregator

```
agunglucker@DESKTOP-5LGBP8F:/mnt/c/Users/asus/OneDrive/Documents/SEMESTER-7-IF/AAT/Tubes-IF4031-Kel12_JuliusAAT$ docker compose exec
client-service wget -qO- "http://aggregator:8090/hazard-events"
{"data":[{"hazard_id":"BMKG-SE661","source":"BMKG","source_ref_id":"SE661","hazard_type":"SEISMIC","severity":"NORMAL","area_name":"J
awa Timur","latitude":-8.04,"longitude":111.92,"occurred_at":"2026-10-08T18:32:49.742Z","ingested_at":"2026-10-08T18:32:50.951Z","att
ributes":{"depth_km":100,"magnitude":3.35,"potential_tsunami":false}},{"hazard_id":"BMKG-SE662","source":"BMKG","source_ref_id":"SE66
2","hazard_type":"SEISMIC","severity":"NORMAL","area_name":"Jawa Barat","latitude":-6.89,"longitude":108.42,"occurred_at":"2026-10-08
T18:32:50.800Z","ingested_at":"2026-10-08T18:32:50.954Z","attributes":{"depth_km":28,"magnitude":3.12,"potential_tsunami":false}},{"h
azard_id":"PVMBG-VR648","source":"PVMBG","source_ref_id":"VR648","hazard_type":"VOLCANIC","severity":"WASPADA","area_name":"Gunung Ij
en","latitude":-8.058,"longitude":114.242,"occurred_at":"2026-10-08T18:32:56.925Z","ingested_at":"2026-10-08T18:32:59.937Z","attribut
es":{"eruption_count_24h":8,"ash_column_height_m":483}},{"hazard_id":"PVMBG-VR649","source":"PVMBG","source_ref_id":"VR649","hazard_t
ype":"VOLCANIC","severity":"SIAGA","area_name":"Gunung Bromo","latitude":-7.9425,"longitude":112.953,"occurred_at":"2026-10-08T18:32:
58.931Z","ingested_at":"2026-10-08T18:32:59.940Z","attributes":{"eruption_count_24h":2,"ash_column_height_m":819}},{"hazard_id":"BMKG
-SE663","source":"BMKG","source_ref_id":"SE663","hazard_type":"SEISMIC","severity":"WASPADA","area_name":"Sumatera Barat","latitude":
-0.91,"longitude":100.46,"occurred_at":"2026-10-08T18:32:59.865Z","ingested_at":"2026-10-08T18:33:00.027Z","attributes":{"depth_km":1
44,"magnitude":6.16,"potential_tsunami":false}},{"hazard_id":"PVMBG-VR650","source":"PVMBG","source_ref_id":"VR650","hazard_type":"VO
LCANIC","severity":"WASPADA","area_name":"Gunung Semeru","latitude":-8.108,"longitude":112.922,"occurred_at":"2026-10-08T18:33:08.018
Z","ingested_at":"2026-10-08T18:33:08.873Z","attributes":{"eruption_count_24h":17,"ash_column_height_m":131}},{"hazard_id":"BMKG-SE66
4","source":"BMKG","source_ref_id":"SE664","hazard_type":"SEISMIC","severity":"AWAS","area_name":"Jawa Tengah","latitude":-7.81,"long
itude":110.37,"occurred_at":"2026-10-08T18:33:08.938Z","ingested_at":"2026-10-08T18:33:09.018Z","attributes":{"depth_km":1,"magnitude
":6.54,"tsunami_warning":{"warning_id":"TW080","threat_level":"Awas","affected_zones":["Jawa Tengah"],"related_event_id":"SE664","est
imated_arrival":"2026-10-08T19:03:08.938Z"},"potential_tsunami":true}},{"hazard_id":"PVMBG-VR651","source":"PVMBG","source_ref_id":"V
R651","hazard_type":"VOLCANIC","severity":"AWAS","area_name":"Gunung Sinabung","latitude":3.17,"longitude":98.392,"occurred_at":"2026
-10-08T18:33:18.060Z","ingested_at":"2026-10-08T18:33:20.949Z","attributes":{"eruption_count_24h":4,"ash_column_height_m":3448}},{"ha
zard_id":"BMKG-SE665","source":"BMKG","source_ref_id":"SE665","hazard_type":"SEISMIC","severity":"NORMAL","area_name":"Jawa Tengah","
latitude":-7.81,"longitude":110.37,"occurred_at":"2026-10-08T18:33:18.983Z","ingested_at":"2026-10-08T18:33:21.051Z","attributes":{"d
epth_km":106,"magnitude":3.8,"potential_tsunami":false}},{"hazard_id":"PVMBG-VR652","source":"PVMBG","source_ref_id":"VR652","hazard_
type":"VOLCANIC","severity":"AWAS","area_name":"Gunung Merapi","latitude":-7.5407,"longitude":110.4457,"occurred_at":"2026-10-08T18:3
3:25.118Z","ingested_at":"2026-10-08T18:33:26.953Z","attributes":{"eruption_count_24h":17,"ash_column_height_m":3753}},{"hazard_id":"
```

2.4.5. Jawaban atas butir "Di laporan"

Sistem dipecah berdasarkan perbedaan siklus perubahan dan kepemilikan data. Fungsionalitas yang berubah bersama dan berbagi data dikelompokkan dalam satu service, misalnya Aggregator yang memuat polling, pemetaan skema, dan penulisan ke Canonical Store. Sebaliknya, fungsionalitas autentikasi client beserta datanya dipisahkan ke service

tersendiri. Dengan begitu, perbaikan pada autentikasi hanya men-deploy ulang token service tanpa menyentuh pemetaan skema yang tidak berubah.

Setiap penyimpanan dimiliki tepat oleh satu service, sehingga service lain mengakses data melalui API pemiliknya. Dengan demikian, tidak ada satu service besar yang memegang akses ke banyak database. Pemisahan ini juga memudahkan pengujian tiap service secara terpisah dan membatasi dampak kegagalan, karena matinya satu service tidak menjatuhkan service lain yang tidak bergantung padanya.

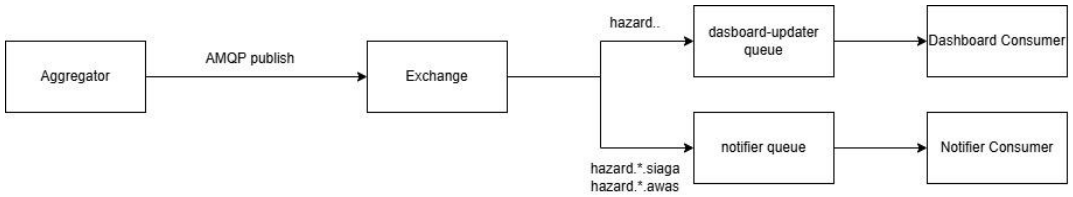
Dua alternatif pembagian lain tidak dipilih. Pendekatan monolith dalam satu container besar membuat perubahan kecil sekalipun mengharuskan seluruh sistem di-build dan di-deploy ulang, dan satu kegagalan menjatuhkan seluruh sistem. Pemecahan per fungsionalitas menghasilkan terlalu banyak service, sehingga menambah panggilan jaringan serta membuat pemeliharaan dan pengujian jauh lebih kompleks.

Perbandingan tiga alternatif penyimpanan (MongoDB, PostgreSQL, dan key-value store) beserta kelebihan dan kekurangannya disajikan pada tabel Keputusan di atas. PostgreSQL dipilih karena paling sesuai dengan dua kebutuhan utama sistem, yaitu ID berurutan otomatis pada service PVMBG dan BMKG, serta fleksibilitas field tambahan melalui JSONB. PostgreSQL memenuhi kedua kebutuhan tersebut secara native dalam satu produk. MongoDB sebenarnya dapat memenuhi kebutuhan ID berurutan melalui koleksi counter dengan $inc, tetapi cara itu menambah kerja manual dan constraint di level basis datanya lebih longgar. Key-value store unggul pada kecepatan baca dan tulis, tetapi tidak mendukung query berdasarkan waktu atau sumber secara native, sehingga indeks harus dibangun dan dijaga secara manual.

Konsekuensi dari pilihan ini adalah validitas isi JSONB sepenuhnya bergantung pada kontrak dengan sumber data, karena tidak divalidasi di level basis data. Selain itu, query ke field di dalam JSONB perlu dioptimalkan dengan indeks GIN atau expression index bila volume data bertambah. Dari sisi ketersediaan, Canonical Store tidak direplikasi sehingga bila store mati, Aggregator tidak dapat menulis maupun melayani data.

2.5. Fan-out Event Distribution/Pub-sub

2.5.1. Rancangan Solusi



Aggregator menghasilkan `HazardEvent` yang perlu diterima oleh beberapa consumer, antara lain dashboard internal dan notifier. Jika Aggregator memanggil setiap consumer secara langsung, producer akan mengetahui seluruh consumer dan consumer yang lambat dapat menghambat consumer lain.

Solusi yang dipilih adalah message broker dengan pola pub/sub dengan aggregator menjadi producer, RabbitMQ menjadi broker serta dashboard dan notifier menjadi consumer

Aggregator membuat sebuah topik bernama hazard.events pada exchange. Routing pada exchange dilakukan berdasarkan tipe bahaya dan tingkat severity dengan format hazard.<hazard_type>.<severity>, contohnya hazard.seismic.awas atau hazard.volcanic.siaga. Aggregator cukup mengetahui exchange tujuan dan tidak perlu mengetahui queue maupun identitas consumer yang berlangganan.

2.5.2. Alasan keputusan

Alternatif	Kelebihan	Kekurangan
HTTP call langsung	Sederhana dan mudah di-debug	Producer coupled dengan consumer, seperti consumer lambat memengaruhi producer
RabbitMQ topic exchange (dipilih)	Routing fleksibel, queue independen, cocok untuk severity	Replay dan partisi tidak sekuat platform log terdistribusi
Kafka	Throughput tinggi dan	Lebih berat dan implementasi saat ini tidak butuh adanya replay

	replay historis kuat	
Satu queue bersama	Konfigurasi sederhana	Consumer saling berebut pesan dan bukan fan-out independen

2.5.3. Batasan

2.5.3.1. Tidak ada dead-letter exchange khusus yang didefinisikan pada konfigurasi saat ini.

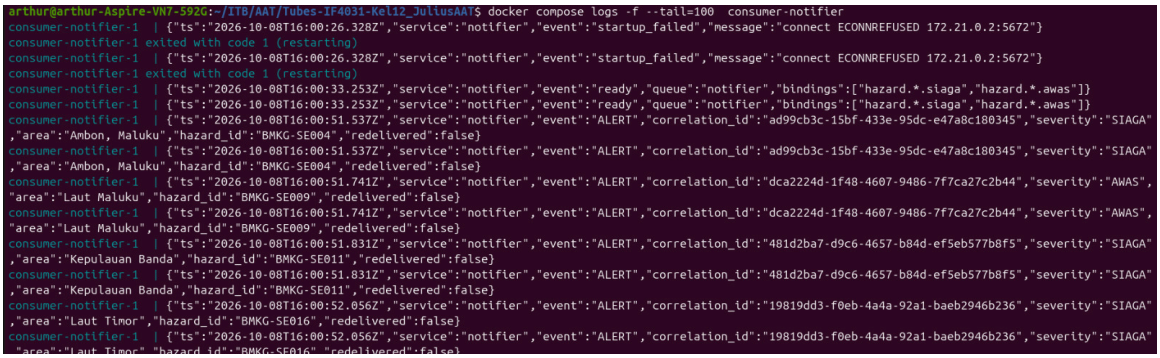
2.5.3.2. Consumer baru perlu mengikuti kontrak JSON HazardEvent.

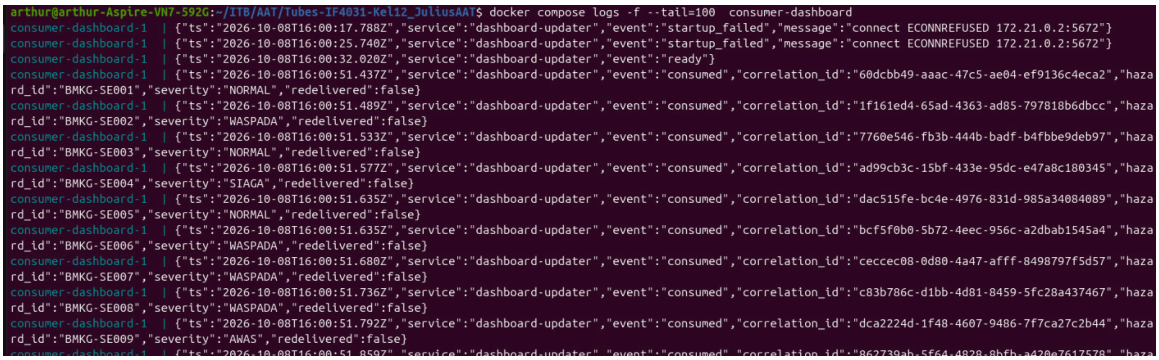
2.5.3.3. Deduplikasi pada consumer tidak *durable.* Set event hanya disimpan pada memori.

2.5.4. Bukti Penyelesaian

2.5.4.1. Dua consumer menerima event dari Producer (Aggregator)

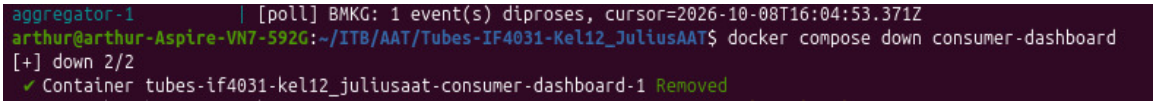


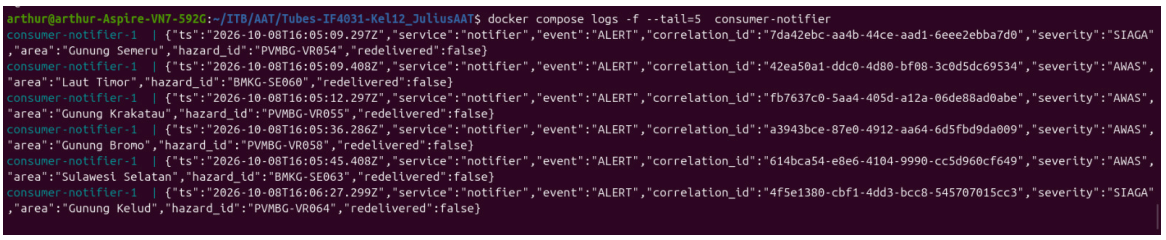


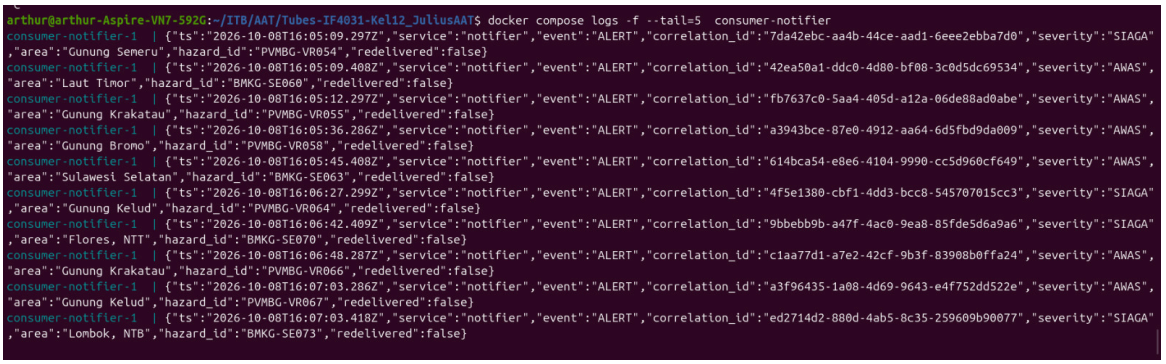


2.5.4.2. Mematikan salah satu consumer

Dilakukan compose down pada consumer dashboard. Dapat terlihat, consumer- notifier tetap mendapatkan update secara normal.



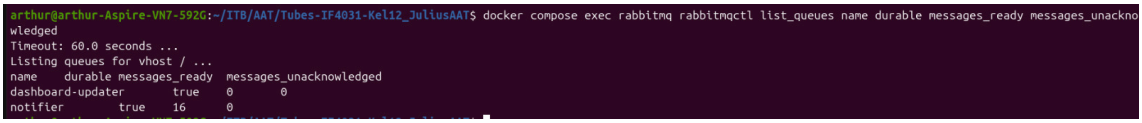




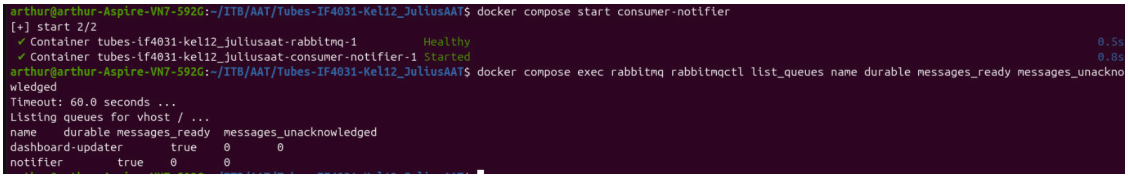
Sekarang, dilakukan compose stop pada consumer notifier.



Exchange rabbitmq tetap mengirim pesan ke queue consumer-notifier. Dapat terlihat, terjadi penumpukan pesan pada queue notifier.

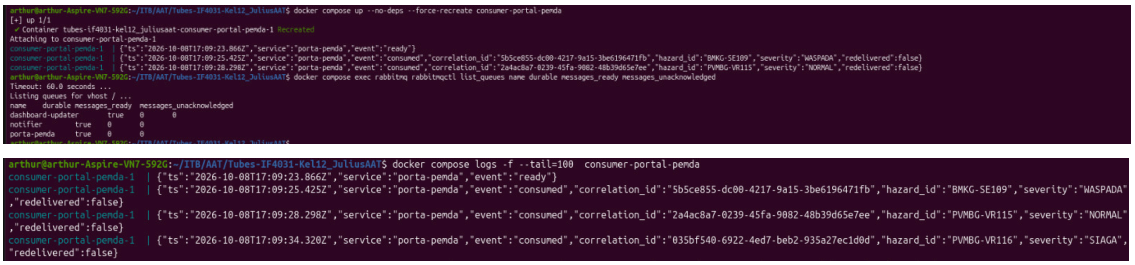


Setelah container consumer-notifier kembali dinyalakan, 16 pesan tersebut dikonsumsi secara normal. Consumer tetap mendapat pesan yang dihasilkan saat consumer tersebut sedang tidak aktif.



2.5.4.3. Menambahkan consumer baru

Ditambahkan sebuah consumer baru bernama consumer-portal-pemda. Penambahan consumer hanya dilakukan dengan menambahkan sebuah container baru yang terhubung ke exchange rabbitmq yang sama tanpa mengubah kode producer. Dapat terlihat, terdapat sebuah queue baru untuk consumer tersebut.



2.5.5. Jawaban atas butir "Di laporan"

Broker yang dipilih adalah RabbitMQ dengan topic exchange durable bernama hazard.events. Aggregator mempublikasikan event dengan routing key hazard.<hazard_type>.<severity>, sedangkan dashboard dan notifier memiliki queue durable serta binding masing-masing. Dengan demikian, producer hanya mengetahui exchange dan tidak mengetahui daftar consumer.

Pola pengiriman yang ditargetkan adalah at-least-once delivery. Pesan dibuat persistent, consumer melakukan acknowledgement manual setelah pemrosesan, dan pesan yang belum di-ack dapat dikirim ulang. Konsekuensinya, consumer harus idempoten. Dashboard melakukan overwrite berdasarkan hazard_id, sementara notifier menggunakan hazard_id:severity sebagai kunci deduplikasi selama proses berjalan.

Saat notifier dihentikan, producer tetap mempublikasikan event dan dashboard tetap mengonsumsi event melalui queue berbeda. Pesan notifier yang sudah berada di queue durable dapat diproses setelah notifier hidup kembali. Consumer baru dapat ditambahkan dengan queue dan binding baru tanpa perubahan pada kode Aggregator.

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM :

1. Muhammad Aufa Farabi / 13523023
2. Julius Arthur / 13523030
3. Ferdinand Gabe Tua Sinaga / 13523051

Fakultas/Sekolah : STEI
Kode Mata Kuliah : IF4031
Nama Mata Kuliah : Aplikasi Arsitektur Terdistribusi
Judul Tugas/Proposal : Milestone 1 Tugas Besar IF4031 Arsitektur Aplikasi Terdistribusi

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah: Claude

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	Ringan

Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	Ringan
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung 10 Oktober 2026



Muhammad Aufa Farabi, 13523023

Bandung 10 Oktober 2026



Julius Arthur, 13523030

Bandung 10 Oktober 2026



Ferdinand Gabe Tua Sinaga, 13523051